

# IOL - Dossier de sécurité production

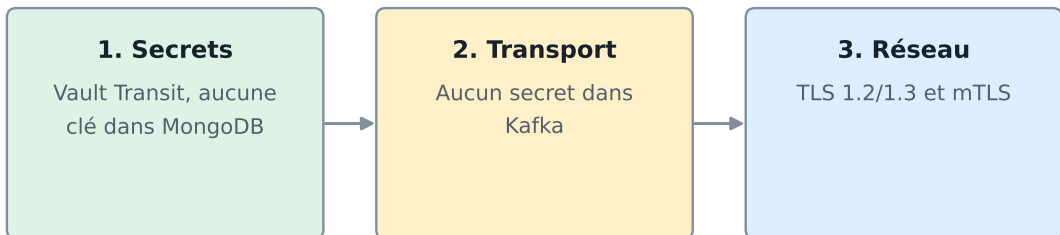
Chiffrement des identifiants métier et TLS entre les composants

---

Décision

## NO-GO en l'état actuel

La production est bloquée tant que les mots de passe JDBC peuvent être persistés en clair, que le mot de passe de destination traverse Kafka et que les liaisons internes sensibles restent non chiffrées.



### Finalité du dossier

- Fixer les décisions d'architecture qui ne sont pas négociables.
- Décrire les scénarios métier standard et big data.
- Organiser une migration sans retour au clair ni au HTTP.
- Définir les tests bloquants et les preuves d'audit.

Version 1.0

Architecture • Développement • Exploitation • Sécurité

# 1. Décisions non négociables

Le secret est protégé pendant tout son cycle de vie, pas seulement dans l'interface

## Décisions

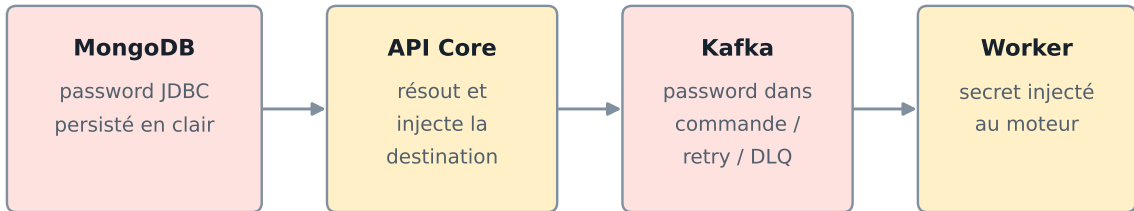
- Les mots de passe utilisateurs restent hachés avec BCrypt.
- Les identifiants métier réutilisables sont chiffrés par Vault Transit.
- La clé de chiffrement n'est jamais dans MongoDB, Git, une image ou une variable d'environnement.
- Kafka ne transporte aucun mot de passe, token, clé privée ou clé d'API.
- Le worker obtient le credential de destination au dernier moment par mTLS.
- Les identifiants dynamiques par exécution sont préférés quand le SGBD les supporte.
- TLS 1.2 minimum, TLS 1.3 préféré, vérification de chaîne et de nom obligatoire.
- Une panne Vault ou TLS échoue de façon fermée, sans fallback en clair.

## Choix technologiques

| Besoin               | Choix                                   | Propriété attendue                 |
|----------------------|-----------------------------------------|------------------------------------|
| Chiffrement          | Vault Transit                           | Clé non exportée, rotation, rewrap |
| Certificats internes | Vault PKI                               | Identité par service, durée courte |
| Certificat public    | ACME / CA publique                      | Reconnu par les navigateurs        |
| Secrets déployés     | Vault Agent / <code>/run/secrets</code> | Pas de variable d'environnement    |
| Kafka                | mTLS + ACL                              | Identité et moindre privilège      |
| SGBD                 | TLS verify-full                         | Confidentialité + anti-usurpation  |

## 2. Risques constatés dans le dépôt

Le masquage dans l'API ne protège ni MongoDB, ni Kafka, ni les backups



### Constats de code

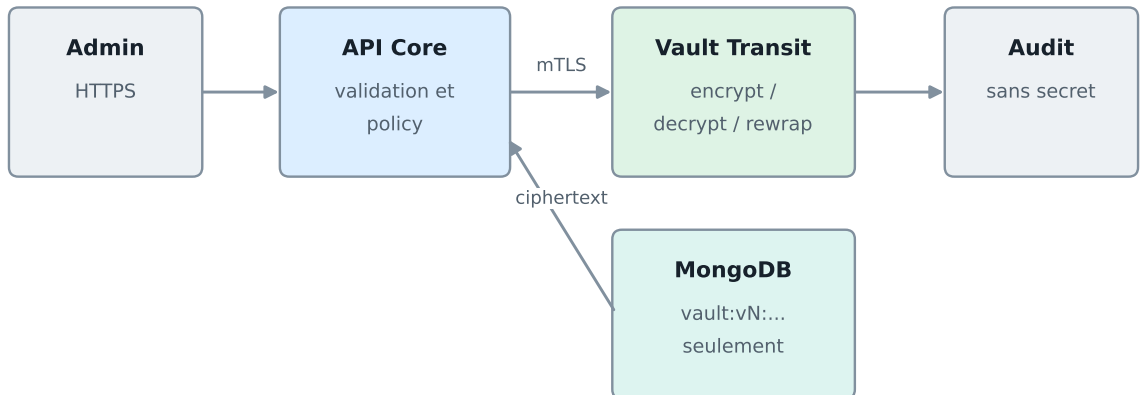
| Élément          | État actuel                 | Niveau               |
|------------------|-----------------------------|----------------------|
| Utilisateur IOL  | BCryptPasswordEncoder       | Correct              |
| Destination JDBC | Champ password persistant   | Critique             |
| Source JDBC / S3 | Champs password / secretKey | Critique si utilisés |
| Commande Kafka   | target_connection.password  | Critique             |
| SQL Server       | encrypt=false dans URL JDBC | Critique             |
| Réseau interne   | HTTP / Kafka PLAINTEXT      | Critique             |

### Conséquence

**Une lecture de backup MongoDB, un accès historique à Kafka, une DLQ ou une interception du réseau interne peut exposer des identifiants réels. La production reste interdite jusqu'à suppression de ces chemins.**

### 3. Architecture cible des secrets

MongoDB conserve le ciphertext ; le secret n'est délivré que pour une exécution autorisée



#### Modèle persistant

##### credential

provider=VAULT\_TRANSIT • keyName=iol-prod-business-credentials •  
ciphertext=vault:v3:... • keyVersion=3 • schemaVersion=1

#### Chiffrement authentifié

AES-256-GCM et donnée associée stable : environment | tenantId | connectionId | purpose.

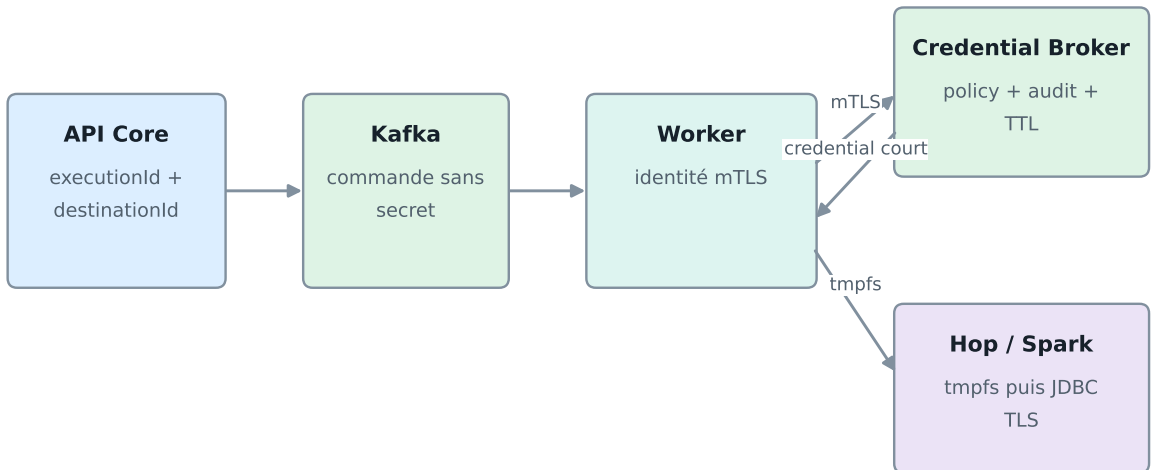
Copier un ciphertext sur une autre connexion doit échouer.

#### Contrat d'API

- La lecture expose passwordConfigured=true/false, jamais \*\*\* ni le ciphertext.
- Conserver l'ancien secret est une commande explicite.
- Une erreur Vault refuse l'écriture ; aucune sauvegarde en clair.

## 4. Exécution sans secret dans Kafka

La commande transporte une référence ; le worker s'authentifie au dernier moment



### Commande Kafka autorisée

#### Payload

```
{"executionId":"...", "workflowId":"...", "destinationConnectionId":"...", "transport":{"mode":"KAFKA|RUSTFS"}}
```

### Conditions de délivrance

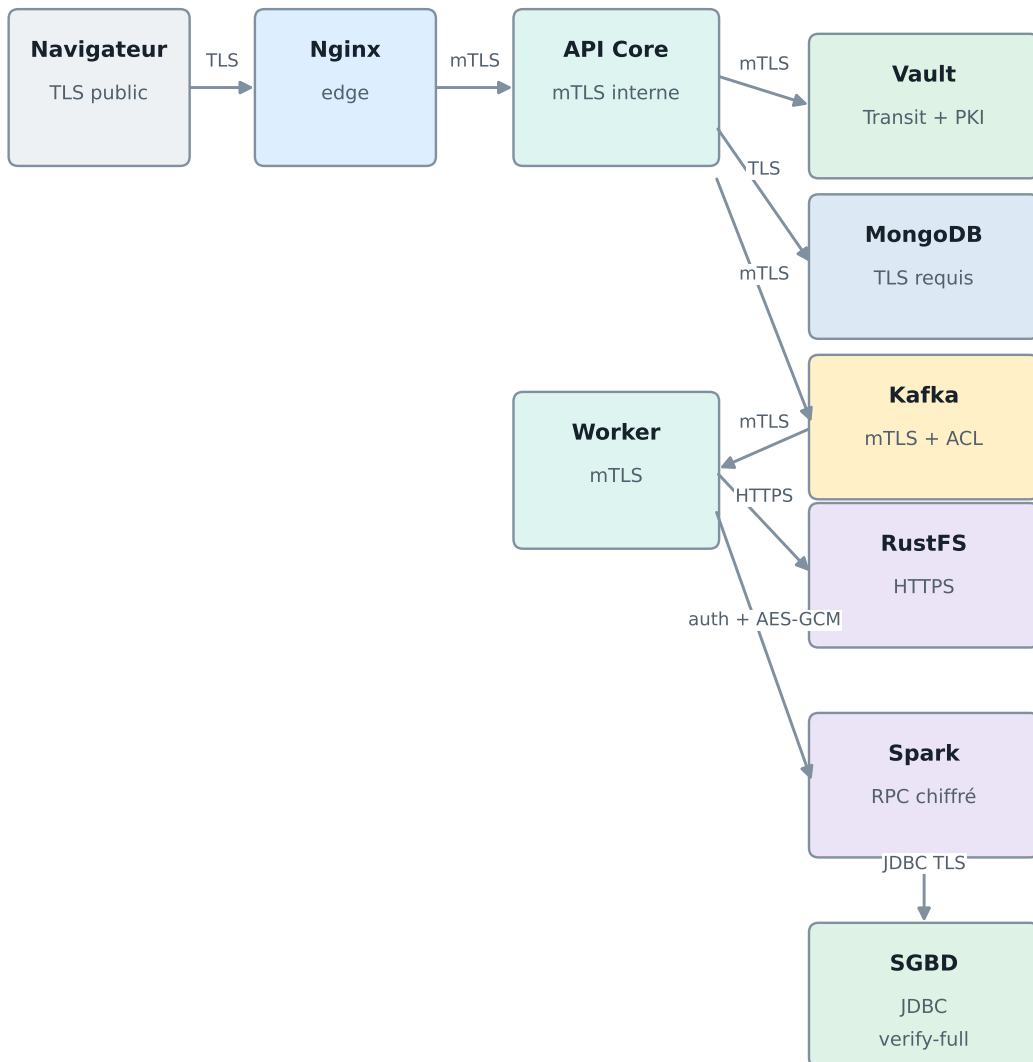
- Certificat client iol-pipeline-consumer valide.
- Exécution RUNNING et destination identique à l'enregistrement.
- Fenêtre temporelle, rate limit, audit et Cache-Control: no-store.
- Secret dynamique par exécution ou secret statique déchiffré au dernier moment.
- Aucune persistance worker ; suppression du tmpfs à la fin.

### Interdictions

**Pas de secret dans les arguments, JSON de workflow, variables globales, logs Hop/Spark, historique, métriques, retry ou DLQ.**

## 5. Topologie TLS et mTLS

Une chaîne de confiance complète, du navigateur jusqu'aux moteurs et bases



### Règles PKI

- Certificat et clé distincts par service ou instance ; SAN DNS obligatoire.
- Certificats internes courts, renouvelés automatiquement à 1/3 du TTL.
- Clés privées en lecture seule dans /run/secrets, jamais dans l'image.
- Ancienne et nouvelle CA coexistent pendant une rotation contrôlée.

## 6. Matrice de sécurité par composant

Le protocole, l'identité et la validation attendue sont testés séparément

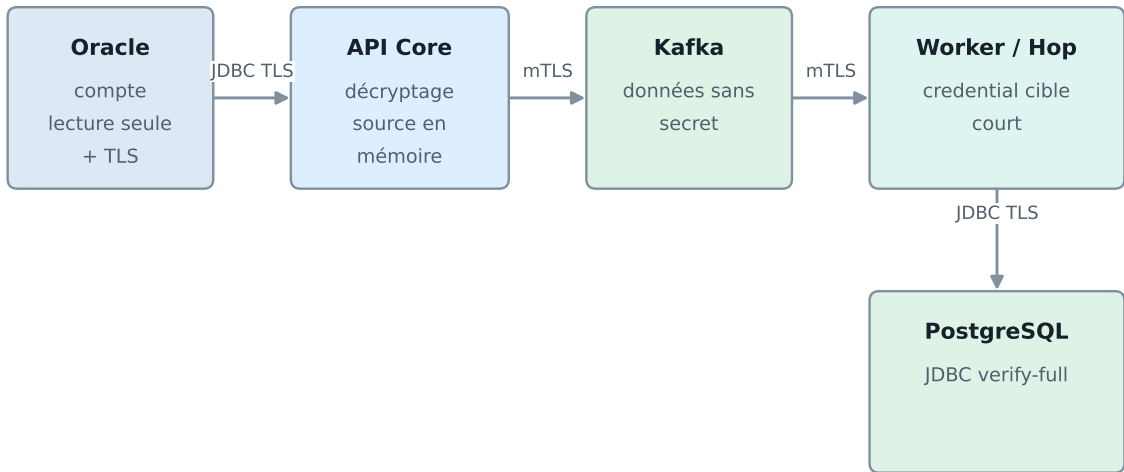
| Client → serveur     | Protection         | Identité / contrôle            |
|----------------------|--------------------|--------------------------------|
| Navigateur → Nginx   | TLS 1.2/1.3        | CA publique + auth applicative |
| Nginx → API Core     | mTLS               | Certificat client Nginx        |
| API Core → Vault     | mTLS               | Workload policy minimale       |
| API Core → MongoDB   | TLS requis         | Compte dédié / X.509           |
| API/Worker → Kafka   | mTLS               | Principal distinct + ACL       |
| Worker → API interne | mTLS               | Certificat pipeline-consumer   |
| API/Worker → RustFS  | HTTPS              | CA vérifiée + policy S3        |
| Spark interne        | Auth + RPC AES-GCM | Secret monté par fichier       |
| Moteur → SGBD        | JDBC verify-full   | Compte dédié/dynamique         |
| Mediator → OpenHIM   | mTLS               | Certificat médiateur           |
| API Core → SMTP      | STARTTLS requis    | Compte technique               |

### Configurations critiques

- Kafka : plus aucun listener PLAINTEXT après migration.
- SQL Server : encrypt=true et trustServerCertificate=false.
- PostgreSQL : sslmode=verify-full et règles hostssl.
- MongoDB : requireTLS, auth et refus TLS 1.0/1.1.
- Spark 3.5.7 : spark.authenticate, crypto version 2, AES/GCM et I/O chiffrée.
- RustFS : RUSTFS\_TLS\_PATH et endpoint https://.

# 7. Cas réel : hôpital, Oracle vers PostgreSQL

250 000 lignes par heure, transport normal par Kafka, aucun accès source depuis Hop/Spark



## Scénario

- API Core teste puis chiffre le mot de passe Oracle avec Vault.
- À l'exécution, API Core lit Oracle en fenêtres et referme la connexion.
- Kafka transporte les lignes et la commande sans identifiant.
- Le worker demande le credential PostgreSQL avec son certificat mTLS.
- Hop charge Bronze puis exécute les transformations autorisées.
- Le lease est révoqué et le secret temporaire supprimé.

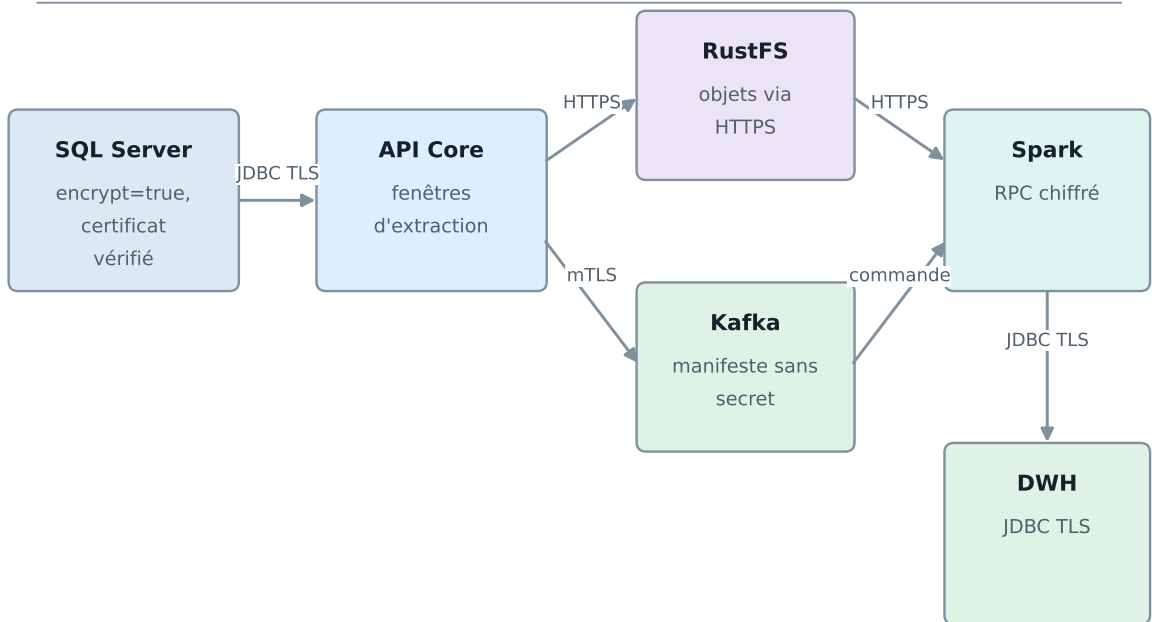
## Preuves de recette

- Zéro occurrence du secret canari dans MongoDB, Kafka, DLQ et logs.
- Capture réseau chiffrée ; mauvais SAN et mauvaise CA refusés.
- Oracle ne peut pas écrire ; PostgreSQL est limité aux schémas ETL.
- Audit relié à executionId, workflowId et identité du worker.



## 8. Cas réel : assurance, SQL Server 2,4 To

Bascule big data automatique ; RustFS porte les données, Kafka porte le manifeste



### Garanties

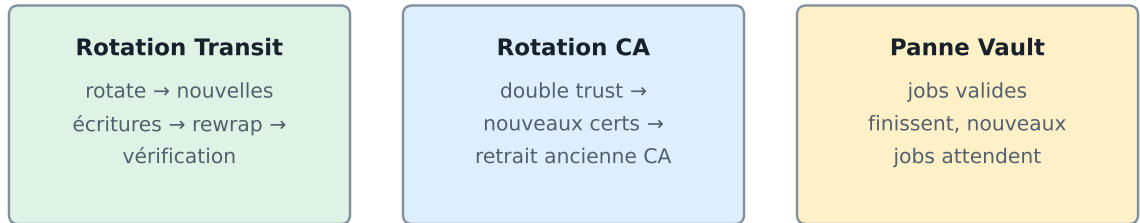
- Hop et Spark ne se connectent jamais directement à la source.
- RustFS reçoit les objets chiffrés en transit et limités au préfixe de l'exécution.
- Kafka conserve uniquement le manifeste signé et les paramètres techniques.
- Le worker obtient la destination par le Credential Broker mTLS.
- Les objets temporaires sont supprimés après succès global et ACK Kafka.
- Une erreur conserve temporairement l'objet pour diagnostic et rejeu selon la rétention.

### Tests volumétriques

**Mesurer le débit Kafka SSL, RustFS HTTPS, Spark chiffré et JDBC TLS. Les seuils de bascule sont validés uniquement avec la sécurité active.**

## 9. Rotations et pannes

La plateforme doit rester sûre quand une clé, un certificat ou Vault change d'état



### Règles de continuité

- Ne relever la version minimale de clé qu'après rewrap complet et preuve de restauration.
- Distribuer le nouveau trust bundle avant les nouveaux certificats.
- Un job déjà lancé peut finir tant que son lease reste valide.
- Aucune nouvelle exécution ne démarre si le credential ne peut être délivré.
- Le retry est borné, idempotent et visible dans le monitoring.
- Aucun secret local de secours, aucun retour HTTP et aucune désactivation de validation.

### Restauration après sinistre

MongoDB et Vault se restaurent comme un couple cohérent. De nouveaux certificats sont émis, les secrets statiques sont rotatés et deux canaris, standard puis big data, valident la reprise.

### Séparation des responsabilités

**Le rôle capable de restaurer MongoDB ne doit pas, seul, restaurer ou administrer les clés Vault. Les opérations critiques exigent un double contrôle.**

# 10. Migration séquencée

Chaque phase possède un Gate ; aucun rollback ne réactive le stockage clair ou HTTP



## Ordre TLS d'un composant

Émettre → distribuer la confiance → activer le serveur TLS → migrer les clients → observer → fermer le listener en clair.

# 11. Recette bloquante

Le succès fonctionnel ne suffit pas : les secrets, le réseau et les scénarios d'échec sont inspectés

| Test                      | Résultat requis                                  |
|---------------------------|--------------------------------------------------|
| Workflow standard         | Données Kafka, zéro secret, destination TLS      |
| Workflow big data         | RustFS HTTPS, manifeste Kafka sans secret        |
| Retry / DLQ               | Reprise idempotente, aucune donnée sensible      |
| Vault indisponible        | Échec contrôlé, aucune sauvegarde/fallback clair |
| Certificat révoqué        | Kafka et Credential Broker refusent le worker    |
| Mauvaise CA / mauvais SAN | Connexion refusée                                |
| Rotation Transit          | Nouvelles écritures + rewrap + restauration      |
| Rotation CA               | Aucune interruption après répétition préprod     |
| Recherche canari          | Zéro occurrence dans tous les supports           |

## Supports inspectés

- MongoDB et backups ; Kafka, retry et DLQ ; logs API, worker, Hop, Spark et OpenHIM.
- Traces, métriques, fichiers temporaires, volumes, variables de conteneur et artefacts CI.
- Capture réseau, ports ouverts, versions TLS, chaîne, SAN, expiration et certificat client.

## Performance

**Les p95 Vault, handshakes, Kafka SSL, RustFS HTTPS, JDBC TLS et temps global sont mesurés avec la sécurité active. Une mesure en clair n'est pas une référence de production.**

# 12. Autorisation de production

Tous les critères GO sont cumulatifs ; un seul NO-GO bloque la bascule

## GO

preuves complètes et recette signée

## NO-GO

un seul défaut critique suffit

### GO obligatoire

- Zéro secret en clair dans MongoDB, Kafka, DLQ, logs, traces et métriques.
- Vault HA, sauvegardé, restauré et soumis au moindre privilège.
- Toutes les liaisons prévues chiffrées avec vérification du nom.
- Kafka sans listener PLAINTEXT ; SQL Server sans encrypt=false.
- Spark authentifié, RPC et données temporaires chiffrés.
- Rotations Transit et CA réussies en préproduction.
- Workflows standard et big data réussis de bout en bout.
- Panne Vault, révocation worker et restauration testées.
- Procès-verbal signé par sécurité, exploitation et propriétaire.

### NO-GO immédiat

- Secret persistant ou présent dans Kafka.
- Validation de certificat désactivée ou port sensible en clair.
- Clé privée dans Git, une image ou une variable d'environnement.
- Fallback automatique vers HTTP ou mot de passe en clair.
- Rotation ou restauration non testée ; défaut critique/haut ouvert.

# 13. Travaux à engager dans le dépôt

Priorité : éliminer l'exposition des secrets avant le basculement TLS global

| Zone                         | Travail principal                                |
|------------------------------|--------------------------------------------------|
| DestinationConnection        | Remplacer password par credential chiffré        |
| SourceMetadata               | Chiffrer JDBC password et S3 secretKey           |
| DestinationConnectionService | Transit + passwordConfigured                     |
| KafkaPipelineEventService    | Supprimer target_connection.password             |
| Credential Broker            | Nouvelle API interne mTLS, TTL, policy, audit    |
| PipelineOrchestrator         | Référence Kafka, credential à l'exécution, tmpfs |
| JDBC                         | Profils TLS ; supprimer encrypt=false            |
| Compose production           | Vault, secrets montés, réseaux et endpoints TLS  |
| Kafka/Mongo/Postgres/RustFS  | Activer TLS et fermer les protocoles clairs      |
| Spark/OpenHIM/SMTP           | Durcissement propre à chaque protocole           |
| CI/CD                        | Secret scan, tests TLS, rotation et canaris      |

## Ordre d'implémentation

- 1. Vault/PKI préproduction.
- 2. Chiffrement et migration des secrets persistés.
- 3. Suppression des secrets dans Kafka.
- 4. TLS/mTLS composant par composant.
- 5. Recette complète, répétition puis canari.

# 14. Références et règle finale

Les choix sont fondés sur les guides officiels des composants présents dans la plateforme

## Références principales

- NIST SP 800-52 Rev. 2 — sélection et configuration de TLS.
- OWASP Cryptographic Storage Cheat Sheet — chiffrement authentifié et cycle de vie des clés.
- HashiCorp Vault — Transit, Database Secrets Engine et PKI rotation primitives.
- Apache Kafka 3.4 — SSL/SASL, authentification et ACL.
- Apache Spark 3.5.7 — authentification, chiffrement RPC, disque local et UI SSL.
- MongoDB — requireTLS et migration roulante vers TLS.
- PostgreSQL — SSL, hostssl et verify-full.
- Spring Boot — SSL Bundles et rechargement des certificats.
- RustFS — RUSTFS\_TLS\_PATH et endpoint HTTPS.
- Docker Compose — secrets montés par fichier.

## Règle finale

### **La plateforme doit échouer de façon sûre**

Une indisponibilité, une expiration ou une erreur de configuration ne doit jamais réactiver un secret en clair, un transport HTTP, un listener Kafka PLAINTEXT ou une validation de certificat désactivée.

**La production commence après les preuves, pas avant.**